Musekwa Fhatuwani -2544974

Sprint 1 Retrospective

This sprint I worked on building the member dashboard, which is the first page a member sees after logging in. I made sure it displayed the right information for whoever was signed in, like their group details and contribution status. I also wrote the CSS files that were used to style the whole app, so the rest of the team had a consistent design to work with when building their parts.

One challenge I ran into was coordinating with the team. We didn't split the work clearly enough at the start so some people ended up with more than others. It made things a bit uneven and caused some delays in getting things done. Another challenge was getting the dashboard to display the correct information depending on who was logged in. Making sure the right data was being pulled and shown to the right user took more time than I expected and required some back and forth to get right.

If I did this sprint again I would make sure we properly assign tasks to everyone during the planning meeting so there's no confusion about who owns what from the beginning. I would also start earlier on the trickier parts instead of underestimating how long they would take.

My work fits into the overall system because the dashboard is basically the home base of the app. Every feature we add going forward like contributions, meetings and payouts will show up through the UI I built this sprint. The CSS files also give the whole app a consistent look so the team doesn't have to redo styling each time.

Musekwa Fhatuwani – 2544974

Sprint 2 Retrospective

During this sprint, I worked on building the invite acceptance flow for the platform. This involved creating the *accept-invite.html* page along with its corresponding JavaScript file. Additionally, I implemented redirect logic to ensure that users are directed to the correct dashboard after accepting an invite, with members going to the member dashboard and treasurers to the treasurer dashboard. By the end of the sprint, the entire flow was functioning end-to-end.

The most challenging part of this sprint was implementing the redirect logic. The system needed to correctly identify the role of the invited user and route them accordingly after acceptance. Ensuring that this role information was properly passed through and triggered the correct redirect required more time than expected and involved several iterations to resolve.

If I were to approach this sprint again, I would spend more time planning the redirect logic upfront before beginning implementation. Clearly defining how role information is passed and handled early on would have reduced the need for rework and made the development process more efficien

Musekwa Fhatuwani – 2544974

Individual Sprint Retrospective — Sprint 3

This sprint I was responsible for building the Analytics & Reports page, covering both the HTML structure (analytics.html) and the full JavaScript module (analytics.js). My contributions included the group selector logic, all three reports, chart rendering with Chart.js, and the CSV export functionality for each report.

I began by setting up the group selector dropdown, wiring it to getMyGroups() and attaching a change listener so switching groups triggers a full reload of all three reports via loadAllReports(). From there I built out Report 1 — the contribution compliance bar chart — using a grouped bar approach to show on-time percentage, late, and missed counts side by side per member. Report 2 was a filled line chart tracking total ZAR collected per month. Report 3 took a different approach entirely, skipping Chart.js in favour of dynamically rendered HTML — a summary stat row followed by a full payout schedule table with colour-coded status badges.

One challenge that came up during chart development was Chart.js silently stacking new chart instances on top of old ones when the group dropdown changed. The fix was storing each chart instance on the canvas element itself (canvas._chartInstance) and calling .destroy() before every re-render. It was a subtle bug that only showed up on repeated group switching.

I also used Promise.allSettled for the three parallel Supabase fetches in loadAllReports(), which meant a failing query on one report would not break the others — each falls back to an empty array independently and shows its own empty state message rather than crashing the whole page.

The main thing I would revisit in Sprint 3 is the payout table markup — it is currently written inline as a template literal inside renderPayoutTable(), which works but becomes hard to maintain as the table grows. Extracting it into a dedicated render helper would clean that up considerably.

Musekwa Fhatuwani

Student number :2544974

Sprint 4 Retrospective

Each sprint pushed me into a different part of the system. Sprint 1 focused mainly on the frontend, where I built the member dashboard and the shared CSS system that the rest of the team later used. Sprint 2 moved more into application logic. The invite acceptance flow required handling user identity, session state, and role-based routing, which were areas I had not worked on much before.

Sprint 3 introduced more complex state management using Chart.js, as well as handling multiple asynchronous requests with Promise.allSettled. This was done so that one failed request would not stop the others from loading. During this sprint, I also encountered a chart stacking issue that only appeared after switching groups multiple times, which showed the importance of testing repeated interactions instead of only testing the first render.

Sprint 4 expanded the analytics module by adding PDF export support. This introduced a different kind of problem because browser-rendered content does not behave the same way inside a PDF document. The main challenge was understanding that a live Chart.js canvas cannot be inserted directly into a PDF and first needs to be converted into an image format that the PDF library can use. Once I understood this difference, the implementation became much simpler.

One of the biggest lessons from Sprint 4 was that some problems come from misunderstanding the environment rather than writing incorrect code. [A similar lesson appeared in Sprint 3, where the chart stacking issue only became visible after repeated user interaction. This highlighted the importance of understanding what state persists between renders.

Using Promise.allSettled in Sprint 3 changed how I approached asynchronous programming. Instead of only thinking about whether a request succeeds, I also considered how the system should behave if one request fails while others continue working correctly.

Building the shared CSS system in Sprint 1 was one of the first times I had to think about code that other developers would directly depend on. [S1] By Sprint 4, I was extending functionality that I originally built during Sprint 3, which made the importance of clean and maintainable code much more practical and immediate.

Looking back, the project developed across a full feature lifecycle: interface design, authentication, reporting, and exporting functionality. The most valuable lesson from the

project was learning to think carefully about the environment, structure, and maintainability of a system before starting implementation.